第 9 课：多元梯度下降

吴恩达《Machine Learning Specialization》· Course 1 · Week 2 · Lesson 1

承上：第 8 课学会了向量化的模型 `f = w·x + b`。 **这一课**：把**梯度下降**也向量化，用在多元模型上——一行更新所有权重，直到收敛。

核心更新式（向量化）：

$$\mathbf{w} := \mathbf{w} - \alpha \cdot \mathbf{d_w}$$

怎么用：从头往下读，代码格按 `Shift + Enter` 运行。

第 0 步：环境自检

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager
_candidates = ["Microsoft YaHei", "SimHei", "PingFang SC",
               "Noto Sans CJK JP", "Noto Sans CJK SC", "Droid Sans Fallback"]
_available = {f.name for f in font_manager.fontManager.ttflist}
_chosen = [c for c in _candidates if c in _available] or ["DejaVu Sans"]
plt.rcParams["font.sans-serif"] = _chosen
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["savefig.bbox"] = "tight"
print("本次使用中文字体：", _chosen)
print(" 环境就绪，可以继续啦！")
```

```
本次使用中文字体： ['Noto Sans CJK JP', 'Droid Sans Fallback']
 环境就绪，可以继续啦！
```

一、多元梯度下降：每个权重各有一个梯度

和单变量一样，只是 w 变成了 n 个，每个都要有自己的梯度：

$$w_j := w_j - \alpha \frac{\partial J}{\partial w_j}, \quad j = 1, \ldots, n$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

其中偏导是：

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left(f^{(i)} - y^{(i)} \right) x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \left(f^{(i)} - y^{(i)} \right)$$

> **和第 6 课几乎一模一样**！只是多了下标 j——每个特征的梯度，都要乘上它自己的那个特征值 x_j。

向量化写法（一次算出所有 w 的梯度）：

$$\mathbf{d_w} = \frac{1}{m}\sum_{i=1}^{m}\left(f^{(i)} - y^{(i)}\right)\mathbf{x}^{(i)}$$

$$\mathbf{w} := \mathbf{w} - \alpha \cdot \mathbf{d_w}, \quad b := b - \alpha\frac{1}{m}\sum(f^{(i)} - y^{(i)})$$

二、跑起来看：多个特征一起学习

用 2 个特征（面积、卧室数）来预测房价，看梯度下降能不能同时学出两个权重。

In [2]:

```
# 生成房价数据：房价 ≈ 0.1*面积 + 5*卧室 + 30 + 一点噪声
np.random.seed(0)
m = 200
x1 = np.random.uniform(500, 2500, m)      # 面积
x2 = np.random.randint(1, 6, m)           # 卧室
y = 0.1*x1 + 5*x2 + 30 + np.random.randn(m)*20

X = np.column_stack([x1, x2])             # m×2 的特征矩阵
w = np.zeros(2); b = 0.0
alpha = 5e-8
J_hist = []

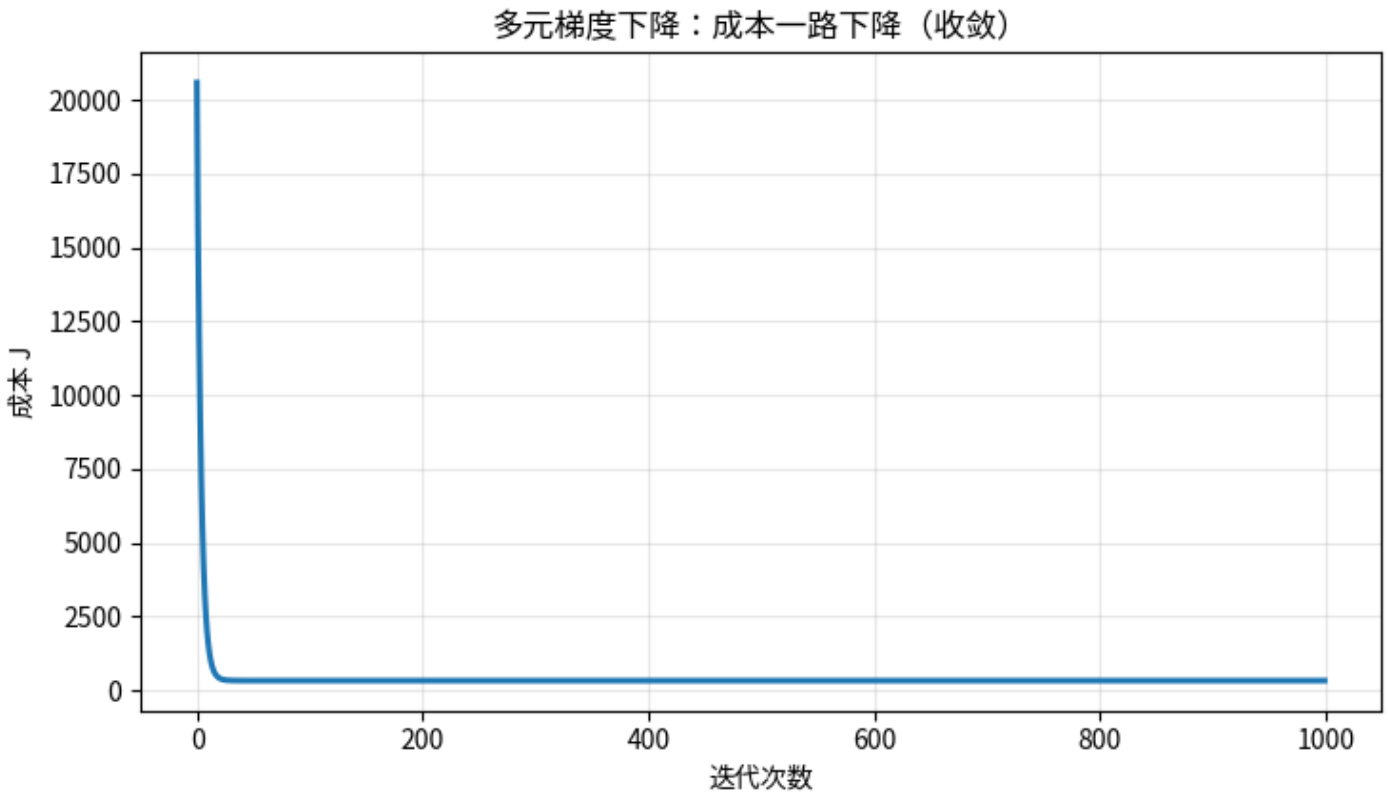
for i in range(1000):
    f = X @ w + b                         # 向量化预测：所有样本一次算完
    err = f - y
    dw = (1/m) * (X.T @ err)              # 向量化梯度：一次算出 w1、w2
    db = (1/m) * np.sum(err)
    w = w - alpha * dw
    b = b - alpha * db
    J_hist.append((1/(2*m)) * np.sum(err**2))

print("学到的 w =", np.round(w, 3), " b =", round(b, 2))
print("真实参数:   w ≈ [0.1, 5]  b ≈ 30")
print("最终成本 J =", round(J_hist[-1], 2))
```

```
学到的 w = [0.125 0.002]  b = 0.0
真实参数:   w ≈ [0.1, 5]  b ≈ 30
最终成本 J = 336.71
```

In [3]:

```
plt.figure(figsize=(7.5, 4.4))
plt.plot(J_hist, color="tab:blue", linewidth=2.2)
plt.xlabel("迭代次数"); plt.ylabel("成本 J")
plt.title("多元梯度下降：成本一路下降（收敛）")
plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
```



看：

- 学到的 `w ≈ [0.1, 5]`、`b ≈ 30`，和真实参数对得上——**它同时学对了两个权重**；
- 成本曲线一路下降，最后稳住。

> 关键就一句：**梯度下降对每个权重分别求梯度、分别更新；向量化让这件事一条公式就做完。**

三、小结

1. **多元梯度下降**：每个 w_j 有自己的梯度 $\partial J/\partial w_j$（误差 × 该特征 x_j），`b` 的梯度还是 `平均误差`。
2. **向量化**：`dw = (1/m) * X.T @ err`，一条公式算出全部权重的梯度。
3. **同时更新** w 的所有分量和 b。
4. **收敛**：J 不再下降就停；多元的碗也是凸的，梯度下降必到全局最优。

四、反转课堂 · 课后作业

1. 为什么 $\partial J/\partial w_j$ 要乘上它自己的特征 $x_j^{(i)}$，而不是别的特征？
2. 代码里 `dw = (1/m) * (X.T @ err)` 这一行，干的是什么？如果用 for 循环写出来是什么样？
3. 多元梯度下降需要**同时更新**所有 w 和 b，为什么这一点这么重要？

下一课（第 10 课）：特征缩放——当特征之间数值差太多时，怎么让梯度下降更快。